

RELAZIONE

Progetto Basi di Dati e Sistemi Informativi

A.A. 2024/2025

Negozio di Carte

Autori:

Matricola: 7119077	Cognome: Pucci	Nome: Andrea
Matricola: 7055273	Cognome: Del Moro	Nome: Alessandro
Matricola: 7109804	Cognome: Blajuti	Nome: Alexandro

Testo del progetto:

La catena di negozi di carte collezionabili "*Ciociamon*" richiede lo sviluppo di un sistema database avanzato per la gestione efficace dei propri negozi e per il coinvolgimento dei membri della community.

Ogni negozio possiede una **Partita Iva**, un **telefono** e un **indirizzo** (composto da Cap Via e Città) in cui lavorano gli impiegati: Un **responsabile**, il quale è registrato con il proprio **ID**, **nome**, **cognome**, **e-mail**, **numero di telefono**, **e-mail**, **stipendio** e la **data dell'inizio del contratto**; Sotto il responsabile si trovano i **cassieri**, registrati con i medesimi parametri e assegnati a uno specifico negozio.

Il sistema è progettato per gestire diverse informazioni essenziali riguardanti i prodotti in vendita, come **carte singole**, **bustine** e **display**, ciascuno identificato da un **ID**, un **nome**, un **prezzo** e il **brand** a cui appartiene.

È fondamentale, inoltre, che il sistema gestisca gli ordini effettuati dai clienti. Ogni ordine è registrato con un **ID unico**, una **data** e il totale dell'**importo**, oltre a dettagli specifici sui prodotti acquistati.

Una delle attività principali della catena di negozi è quella dell'organizzazione di **tornei** ai quali potranno partecipare i **clienti** al fine di creare una community attiva e coinvolta. I **tornei** saranno identificati tramite un **ID** unico, **un nome, una data e un'ora, la quota d'iscrizione e il premio in caso di vincita.**

Un elemento cruciale del sistema è la gestione della community, ovvero dei **clienti** registrati e possedenti una **tessera fedeltà**. Ogni membro sarà registrato tramite un **ID** unico, un **nome** e un **cognome, il codice fiscale e l'indirizzo**; La **tessera** permetterà loro di accumulare punti ogni volta che effettuano un acquisto. La partecipazione ai tornei sarà gestita in modo da registrare i risultati e assegnare premi in base alle performance.

Gli **ordini** e le partecipazioni dei membri sono monitorati per il calcolo dei punti accumulati sulla **tessera fedeltà**, la quale può avere diverse **categorie** (Bronze, Silver, Gold). Ogni **categoria** offre sconti e vantaggi esclusivi all'interno della catena di negozi.

Glossario dei termini

Nome	Descrizione	Termini relativi	Collegamenti
<i>Negozio</i>	Punto vendita della catena di carte collezionabili.	Partita IVA, Telefono, Indirizzo (CAP, Provincia, Via, Città)	Torneo, Ordine
<i>Ordine</i>	Transazione commerciale registrata nel sistema.	ID, Data, Importo	Prodotto, Cliente, Negozio
<i>Impiegato</i>	Persona che lavora all'interno di un determinato negozio può essere	N_dipendente, Nome, Cognome, E-mail, Ruolo, Stipendio, DataInizioContratto	Negozio

	un responsabile o un cassiere.		
<i>Cliente</i>	Persona che lavora in un negozio, con ruoli specifici come cassiere o magazziniere.	CodiceFiscale, Nome, Cognome, Via,Città,Provincia, CAP	Ordine, Torneo, Tessera
<i>Tessera</i>	Strumento per accumulare punti e accedere a vantaggi esclusivi in base alla categoria.	IDTessera, Punti accumulati	Categoria, Cliente
<i>Prodotto</i>	Articolo venduto nei negozi della catena (es. carte, bustine, display).	ID_Prodotto, TipologiaProdotto, NomeProdotto, Disponibilità,Brand, Prezzo	Ordine
<i>Categoria</i>	Livelli della tessera fedeltà con vantaggi crescenti.	Tipo (Bronze, Silver, Gold), Sconto	Tessera
<i>Torneo</i>	Evento competitivo organizzato per coinvolgere i clienti e la community.	ID_Torneo, Nome, Data, Ora, Quotaliscrizione, Premio	Negozi, Cliente

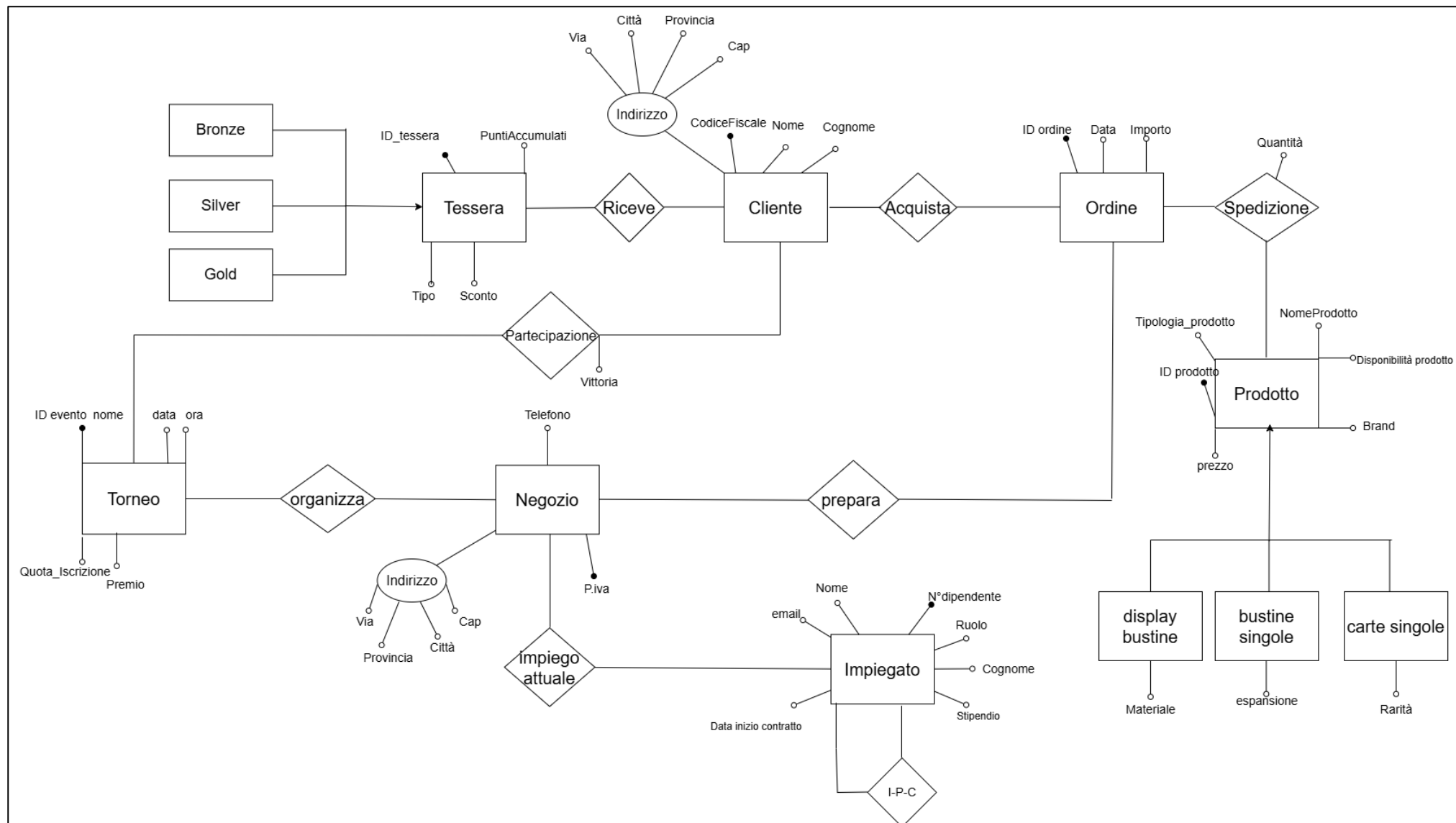
Schema concettuale

Per la realizzazione del nostro schema E-R ci siamo concentrati sul testo della richiesta del cliente esaminando i concetti chiave e provando anticipatamente a tradurre il testo in uno schema Entità Relazioni adeguato.

Siamo partiti dalla gestione dell'entità principale Negozio con i vari collegamenti fino ad arrivare alla gestione degli Ordini e quindi dei Prodotti e dei Clienti.

Un primo schema che abbiamo ottenuto è il seguente:

In questo schema non sono ancora presenti le cardinalità fra le varie relazioni perché in questa prima fase ci siamo concentrati sul dare uno scheletro al progetto su cui, successivamente, potevamo sviluppare idee o migliorie.



Progettazione logica

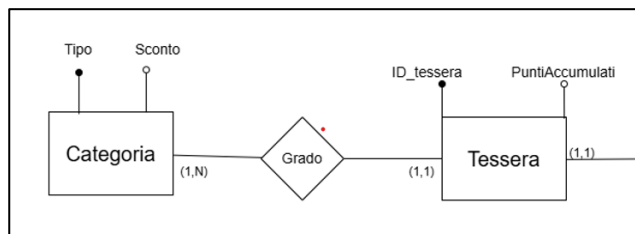
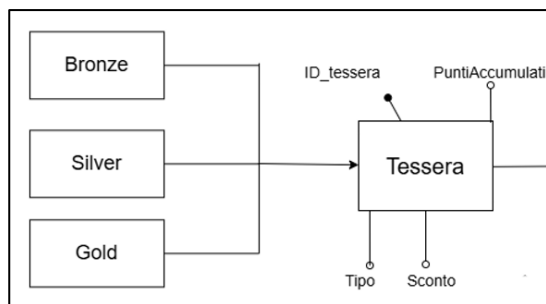
Ristrutturazione schema concettuale

Il nostro approccio è stato abbastanza pratico cercando di analizzare le esigenze del nostro sistema e identificare i casi d'uso più comuni. Abbiamo deciso di mappare le relazioni tra le tabelle in modo che fossero non solo aderenti alla realtà del problema, ma anche facili da gestire e scalabili.

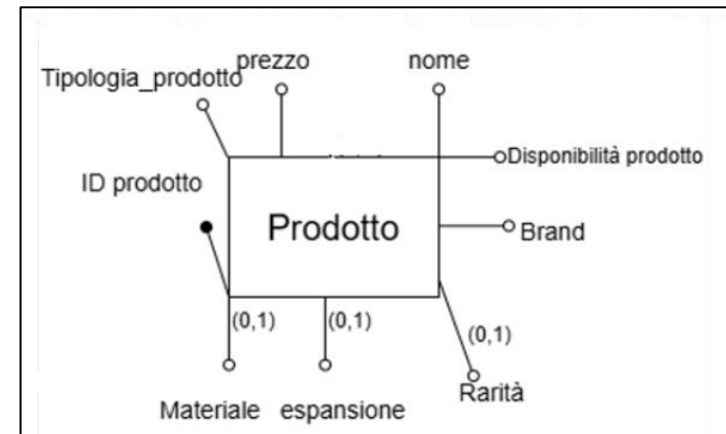
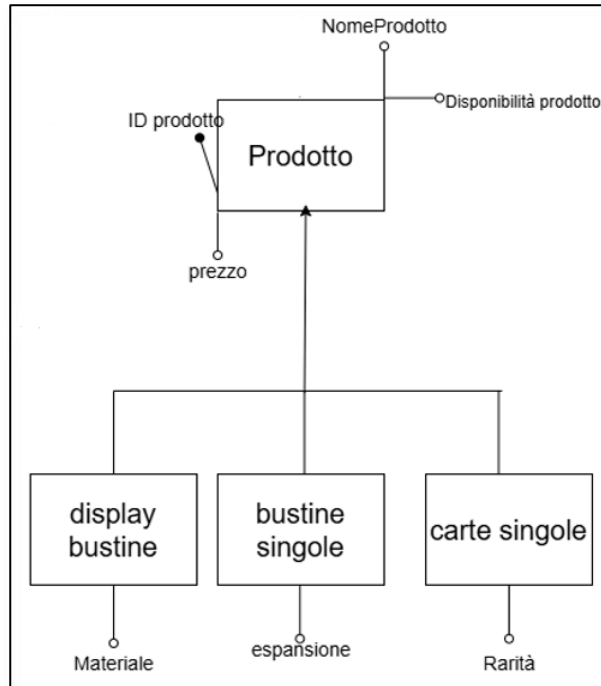
Analisi delle ridondanze

Nello schema non sono state trovate informazioni ridondanti o derivabili da altre.

Eliminazione delle gerarchie



Abbiamo deciso di sostituire la gerarchia in una relazione fra due entità: Tessera e Categoria. Questo approccio è scalabile, in quanto aggiungere una nuova categoria richiede solo un record in Categoria, senza modifiche strutturali. È anche flessibile, permettendo di aggiungere attributi alle categorie senza alterare il modello.



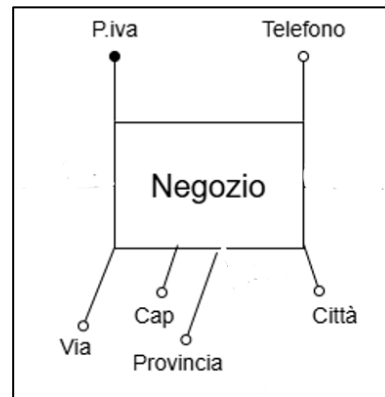
Questa generalizzazione è stata tradotta utilizzando il metodo dell'accorpamento delle figlie nel genitore: gli attributi **Materiale**, **espansione** e **Rarità** adesso sono all'interno di Prodotto ed è stato aggiunto l'attributo "**Tipologia_prodotto**" in modo da distinguere le varie categorie di prodotto.

Eliminazione degli attributi multivalore

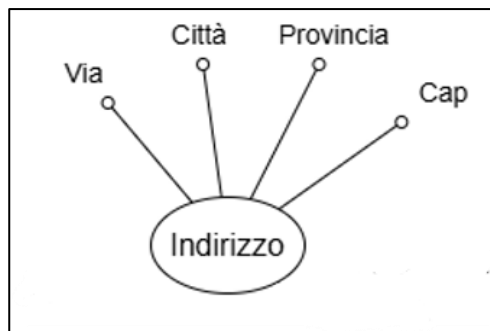
Attributo composto di Negozio



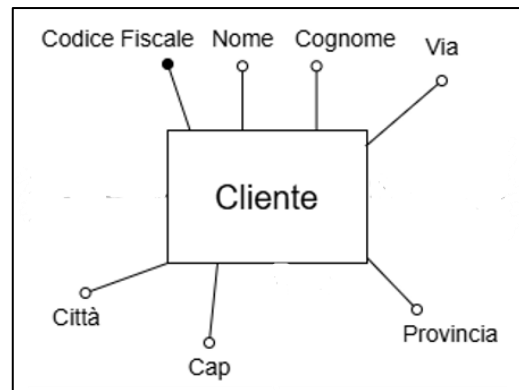
Partizionamento



Attributo composto di Cliente



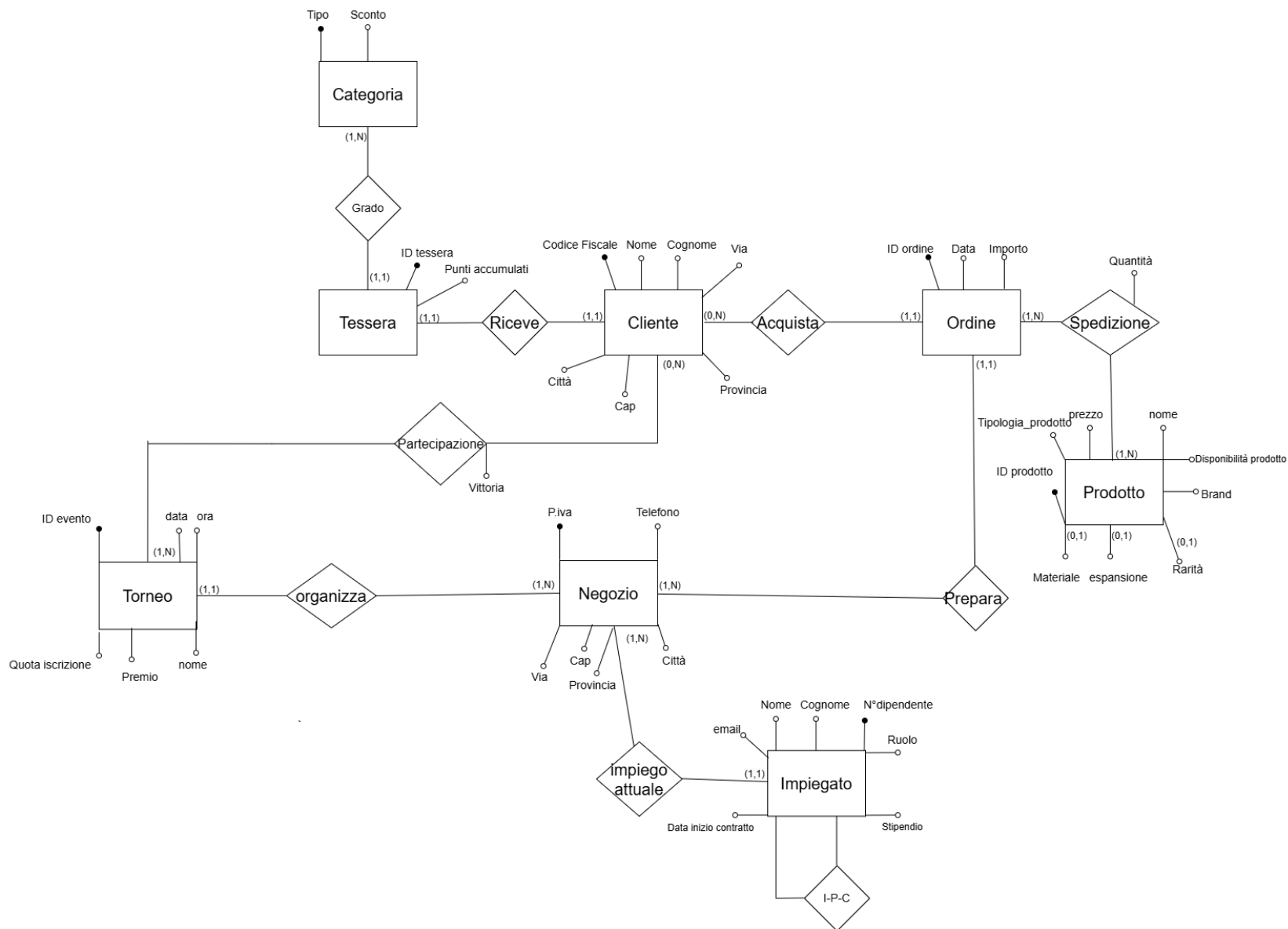
Partizionamento



Scelta degli identificatori primari

In questa fase, ci siamo concentrati sulla scelta degli identificatori primari per ciascuna delle entità nel nostro schema. Inizialmente, avevamo previsto di utilizzare una chiave primaria univoca per ogni entità. Tuttavia, successivamente abbiamo notato che per risolvere le cardinalità tra **Cliente** e **Torneo**, e tra **Ordine** e **Prodotto**, potevamo adottare delle chiavi primarie composte, che includessero le chiavi primarie delle tabelle correlate. Questo approccio ci ha permesso di ottimizzare il database, riducendo la complessità e migliorando l'efficienza, pur mantenendo il rispetto delle specifiche iniziali del progetto.

Schema ER finale



Schema logico

- **Categorie** (Tipo, Sconto)
- **Tessere** (Id tessera, PuntAccumulati, Categoria)
- **Clienti** (CodiceFiscale, Nome, Cognome, Cap, Provincia, Via, Città, Tessera)
- **Ordini** (Id Ordine, Data, Importo, Cliente, Negozio)
- **Spedizioni** (Id Ordine, Id Prodotto, Quantità)
- **Prodotti** (Id Prodotto, Nome, Prezzo, TipologiaProdotto, Brand, Materiale, Espansione, Rarità, DisponibilitàProdotto)
- **Impiegati** (NumeroDipendente, Ruolo, Nome, Cognome, E-mail, Stipendio, Negozio, DataInizioContratto)
- **Negozi** (P_IVA, Via, Città, Cap, Provincia, Telefono)
- **Tornei** (Id Evento, Nome, DataTorneo, Ora, QuotaIscrizione, Premio, Negozio)
- **Partecipazioni** (CodiceFiscale, Id Evento, Vittoria)

Vincoli di integrità referenziale

- **Tessere.Categoria** → **Categorie.Tipo**
- **Clienti.Tessera** → **Tessere.Id_Tessera**
- **Ordini.Cliente** → **Clienti.CodiceFiscale**
- **Spedizioni.Ordine** → **Ordini.Id_Ordine**
- **Spedizioni.Prodotto** → **Prodotti.Id_Prodotto**
- **Ordini.Negozio** → **Negozi.P_IVA**
- **Impiegati.Negozio** → **Negozi.P_IVA**
- **Tornei.Negozio** → **Negozi.P_IVA**
- **Partecipazioni.Cliente** → **Clienti.CodiceFiscale**
- **Partecipazioni.Torneo** → **Tornei.Id_Evento**

Implementazione MySQL

Interrogazioni

Questa è la lista delle interrogazioni realizzate:

1. Trovare i commessi con stipendio maggiore di 17.000 euro.
2. Trovare gli ordini eseguiti dal 2019/01/01 compreso, in ordine crescente.
3. Trovare il numero di partecipazioni ai tornei dei clienti.
4. Trovare Nome e Cognome dei clienti che hanno la tessera Gold.
5. Trovare quali tornei sono stati svolti a Milano.
6. Trovare i prodotti Pokemon con un prezzo maggiore di 10 euro, ordinati in ordine decrescente (utilizzando la vista che permette di visualizzare i prodotti pokémon).
7. Trovare tutte le tessere con 500 o più punti.
8. Trovare la spedizione che contiene più prodotti diversi.

Procedure e funzioni

- Procedura **CapoCommesso** = procedura che, dato un numero_dipendente di un commesso, restituisce nome,cognome,email e stipendio del suo capo.
- Procedura **ClienteVittoriaPremio** = procedura che dato il codice fiscale di un cliente, verifica che abbia partecipato a dei tornei. Se ha partecipato, nel caso di vittoria, la procedura restituisce il nome del torneo e il premio; viceversa la procedura stampa il seguente messaggio 'Il cliente non ha vinto alcun torneo.
- Procedura **DettagliOrdine** = procedura che fornito l'id di un ordine restituisce il prodotto ordinato e in che quantità.
- Procedura **AddNegozio** = procedura che esegue un insert di un nuovo negozio nella catena.

- Procedura **AddImpiegato** = procedura che esegue un insert di un nuovo impiegato nella tabella degli impiegati.
- Procedura **ModificaScontoCategoria** = procedura che permette di modificare la percentuale dello sconto ad una determinata categoria di tessere.

Viste

Per migliorare da un punto di vista concettuale la visualizzazione di alcune tabelle sono state aggiunte le seguenti viste:

- Vista **VistaProdottiPokemon** = vista che consente di visualizzare solo i prodotti di un determinato brand che in questo caso sarà Pokemon.
- Vista **ImpiegatiTorino** = vista che permette di visualizzare tutti gli impiegati appartenenti ad una data sede (in questo caso Torino).
- Vista **Ordini_Torino** = vista che permette di visualizzare tutti gli ordini eseguiti da una determinata sede. (In questo caso Torino)

Trigger

Sono stati aggiunti anche alcuni trigger per migliorare la gestione di modifiche eseguite all'interno del database:

- Trigger **AggiornaPuntiDopoOrdine**= trigger che dopo aver inserito un nuovo ordine nel database, andrà ad aggiungere una certa somma di punti (ovvero 100) alla tessera del cliente. Dopodiché verranno controllati il numero di punti accumulati dal cliente e, se sono maggiori o uguali a 1100, avremo un aggiornamento della tessera al grado superiore che però comporterà l'azzeramento dei punti. L'aggiornamento della tessera seguirà la seguente gerarchia: Bronze→Silver→ Gold.
- Trigger **VerificaDisponibilitàEUpdate**= trigger che dopo aver inserito un nuovo ordine nel database, andrà a verificare l'effettiva disponibilità del prodotto e nel caso ci sia andrà ad aggiornare la corrente disponibilità andando a sottrarre la quantità di prodotto ordinato. Nel caso il prodotto non sia disponibile nella quantità ordinata verrà restituito il seguente messaggio 'Disponibilità del prodotto insufficiente'.
- Trigger **applica_sconto_ordine**= trigger che prima di inserire un nuovo ordine, calcola lo sconto da applicare all'importo speso dal cliente, grazie alla presenza della tessera fedeltà. Dopodiché, va a modificare l'importo iniziale andandolo a sostituire con quello scontato.